COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/02/2026	**DATE SUBMITTED:**	09/02/2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK NO. 7.2 Algorithm Analysis: Best-Case and Worst-Case Time Complexity

1. PROCEDURE OUTPUT

What procedures did you perform?

1. Finding an Element	for (int i = 0; i < n; i++)
	{
	  if (arr[i] == target)
	  {
	  return i;
	  }
	}
		1. Best Case: O(1)
		2. Worst Case: O(N)
		3. Explanation: The for loop iterates N times based on the target element. If the target is the first element, only one iteration is needed. Otherwise, it must go through each element based on N.

2. Printing All Elements	for (int i = 0; i < n; i++)
	{
	  cout << arr[i] << endl;
	}
		1. Best Case: O(N)
		2. Worst Case: O(N)
		3. Explanation: The loop always iterates N times based on the index number.

3. Nested Loops	for (int i = 0; i < n; i++)
	{
	  for (int j = 0; j < n; j++)
	  {
	  cout << i << " " << j << endl;
	  }
	}
		1. Best Case: $O(N^2)$
		2. Worst Case: $O(N^2)$
		3. Explanation: The loop outside the inner loop iterates N times with the loop inside iterates N times, hence NxN -> N^2.

4. Conditional Processing	if (n > 100)
	{
	  for (int i = 0; i < n; i++)
	  {
	  cout << arr[i];
	  }
	}
	else
	{
	  cout << "Small input";
	}
		1. Best Case: O(1)
		2. Worst Case: O(N)
		3. Explanation: The if check is skipped when the number is less than 100, which means it only executes once. If n is more than 100, the number iterates by N times.

5. Fixed Inner Loop	for (int i = 0; i < n; i++) {	1. Best Case: O(N) 2. Worst Case:O(N)

	for (int j = 0; j < 10; j++)
	{
	cout << i << j;
	}
	}
		3. Explanation: The loop outside iterates N times based on the input N, while the inner loop is fixed to iterate 10 times.

6. Two Consecutive Loops	for (int i = 0; i < n; i++)
	{
	cout << arr[i];
	}
	
	for (int j = 0; j < n; j++)
	{
	cout << arr[j] * 2;
	}
		1. Best Case: O(N)
		2. Worst Case:O(N)
		3. Explanation: Two loops concurrently iterate N times. This forms the operations by N + N = 2N which means the time complexity is O(N).

7. Dividing by Two	int x = n;
	
	while (x > 1)
	{
	x = x / 2;
	}
		1. Best Case: O(1)
		2. Worst Case: $O(\log_2 N)$
		3. Explanation:
		If the condition is not met (x > 1), it only iterates once and so, it has a time complexity of O(1). If the condition is met, the value iterates by the number of times based on the values of x divided by 2 until x is no longer greater than 1, hence $O(\log_2 N)$.

8. Nested Loop With Different Limits	for (int i = 0; i < n; i++)
	{
	for (int j = 0; j < i; j++)
	{
	cout << i << j;
	}
	}
		1. Best Case: O(N)
		2. Worst Case: $O(N^2)$
		3. Explanation: When i is set to a zero or negative value, then it will simply iterate by N since the inner loop does not iterate. Otherwise, it iterates by N^2 since the value is positive and non-zero and the nested loop iterates with the outer loop until it meets the condition.

9. Early Termination	for (int i = 0; i < n; i++)
	{
	if (arr[i] == target)
	{
	cout << "Found";
	break;
	}
	}
		1. Best Case: O(1)
		2. Worst Case:O(N)
		3. Explanation: When the target immediately reaches the first element, the iteration only happens once, hence O(1). Otherwise, the loop continues until the target is reached.

10. Two-Level Search	for (int i = 0; i < n; i++) { for (int j = 0; j < n; j++) { if (arr[i] == target) {	1. Best Case: O(N) 2. Worst Case: $O(N^2)$ 3. Explanation: When arr index matches the target, the inner loop breaks and the outer loop continues until its condition is met, which results in O(N). If the target is not found yet, both inner and outer loops will continue

	break; } } }	to I iterate hence N^2.

11. Linear Loop With Logarithmic Inner Work	for (int i = 0; i < n; i++)
	{
	int x = n;
	
	while (x > 1)
	{
	x = x / 2;
	}
	}
		1. Best Case: O(N $\log_2$ N)
		2. Worst Case: O(N $\log_2$ N)
		3. Explanation: The outer loop iterates by N times for each iteration. The inner loop continues to divide by 2 until it no longer satisfies the condition of the value being greater than 1. So, it results in the time complexity of O(N log2 N)..

12. Conditional Nested Loop	if (arr[0] == target)
	{
	cout << "Found";
	}
	else
	{
	for (int i = 0; i < n; i++)
	{
	for (int j = 0; j < n; j++)
	{
	cout << arr[i] + arr[j];
	}
	}
	}
		1. Best Case: O(1)
		2. Worst Case: O(N^2)
		3. Explanation: The best case scenario is reached when the index value immediately matches the target and only performs one execution. Otherwise, it will iterate the inner and outer loops resulting in O(N^2).

13. Finding the Maximum	int max = arr[0];
	
	for (int i = 1; i < n; i++)
	{
	if (arr[i] > max)
	{
	max = arr[i];
	}
	}
		1. Best Case: O(N)
		2. Worst Case: O(N)
		3. Explanation: Because the loop always searches every element of the array from the start, the time complexity is always O(N).

14. Three Nested Loops	for (int i = 0; i < n; i++)
	{
	for (int j = 0; j < n; j++)
	{
	for (int k = 0; k < n; k++)
	{
	cout << i + j + k;
	}
		1. Best Case: O(N^3)
		2. Worst Case: O(N^3)
		3. Explanation: Since there is a nested loop within another nested loop inside the main outer loop, the resulting time complexity is O(N^3).

	} }	

15. Comprehensive Analysis	for (int i = 0; i < n; i++)
	{
	if (arr[i] == target)
	{
	return i;
	}
	
	int x = n;
	
	while (x > 1)
	{
	x = x / 2;
	}
	}
		1. Best Case: O(log N)
		2. Worst Case: O(N log N)
		3. Explanation: If the index matches the target, it continues to a logarithmic loop. Otherwise, it will iterate and then perform the logarithmic process until it meets the condition, hence O(N log N).

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google Gemini, ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation and clarification of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to clarify the process of loops and conditions.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.